Section: 9. Regular Expression

Lecture: 99. Escape Sequences

Section 1: Lecture Summary

This lecture focuses on regular expressions (regex), specifically the use of escape sequences to define patterns. Starting with a recap of regex quantifiers and special characters, the instructor introduces escape sequences beginning with a backslash (\) that represent common character classes such as digits, non-digits, alphanumeric characters, whitespace, and positions within a string. The lecture explains each escape sequence's meaning and usage, comparing them with familiar regex shorthand symbols. Then, it demonstrates how to use these escape sequences combined with quantifiers to build regex patterns for practical examples like date formats, passwords, and email addresses. The lecture emphasizes the foundational knowledge rather than mastery and encourages practicing pattern writing and matching using Python's `re` module.

Section 2: Key Concepts and Explanations

- **Escape Sequences in Regex**

Escape sequences start with a backslash `\` and represent predefined character classes or anchors inside regular expressions:

Escape Sequence	Meaning	Description
`\d`	Digit	Represents a single digit `[0-9]`
`\D`	Non-digit	Any character except digits (letters, special chars, etc.)
`\w`	Alphanumeric (word character)	Letters (lowercase & uppercase), digits, and underscore
`\W`	Non-alphanumeric	Any character not an alphanumeric or underscore (typically

		special characters)
`\s`	Whitespace	Space, tab (`\t`), newline (`\n`), carriage return (`\r`), form feed (`\f`), etc.
`\S`	Non-whitespace	Any character except whitespace
`\A`	Start of string anchor	String must start with the following pattern (similar to `^`)
`\Z`	End of string anchor	String must end with the previous pattern (similar to `$`)

- **Quantifiers**

Used to specify the number of times a pattern or character must occur:

- Exact count: `{n}` (e.g., `{2}` means exactly 2 times)

- Minimum count: `{n,}` (e.g., `{8,}` means 8 or more times)

- Optional: `?` (0 or 1 time)

- One or more: `+`

- Zero or more: ``

- **Combined Usage for Pattern Definitions**

Escape sequences combined with quantifiers enable defining complex patterns like date formats, passwords, and emails, allowing validation by matching strings against these patterns.

- **Special Notes**

- For password patterns, `[\w_]` includes alphanumerics and underscore; combined with quantifiers like `{8,}`, it restricts the minimum size.

- Email patterns require handling alphanumerics, optional dots (`\.`), mandatory `@` symbol, domain names, and specific domain extensions (e.g., `.com`, `.org`).

- Escape sequences help simplify regex writing compared to explicitly stating character ranges or sets.

- Assertions and conditional checks (like restricting date ranges or password complexity rules) are advanced topics for later study.

Section 3: Example Code and Use Cases

Pattern for date in format DD/MM/YYYY

Pattern for password: alphanumeric and underscore, minimum 8 characters

Pattern for a simplified email:

One or more alphanumerics, optional dot (0 or 1), more alphanumerics, '@',

domain name, '.', and domain suffix (com or org), string must end.

```
import re

date_pattern = r'\d{2}/\d{2}/\d{4}'
date_string = "31/12/1970"
match_date = re.match(date_pattern, date_string)
print(f"Date match: {bool(match_date)}")  # True if matches

password_pattern = r'[\w_]{8,}'
password = "abC_1234"
match_password = re.match(password_pattern, password)
print(f"Password match: {bool(match_password)}")  # True if matches

email_pattern = r'\w+\.?\w+@\w+\.(com|org)\Z'
email = "my.id1@gmail.com"
match_email = re.match(email_pattern, email)
print(f"Email match: {bool(match_email)}")  # True or False depending on
pattern and email
```

- The above uses escape sequences:

- `\d` for digits in the date

- `\w` for alphanumeric in password and email

- `\.` to match literal dot in email

- `\Z` to enforce string end anchor in email verification

- Quantifiers `{2}`, `{4}`, `{8,}`, `+`, and `?` are used to specify exact, minimum, and optional occurrences in patterns.

Section 4: Key Takeaways

- Escape sequences in regex simplify pattern definitions by representing common character classes like digits, non-digits, word characters, and whitespace.

- Quantifiers control the number of times these patterns occur, allowing precise pattern construction.

- Anchors `\A` and `\Z` assert the start and end of a string, similar to `^` and `$` respectively.

- Patterns for dates, passwords, and emails can be written by combining escape sequences with quantifiers and special characters.

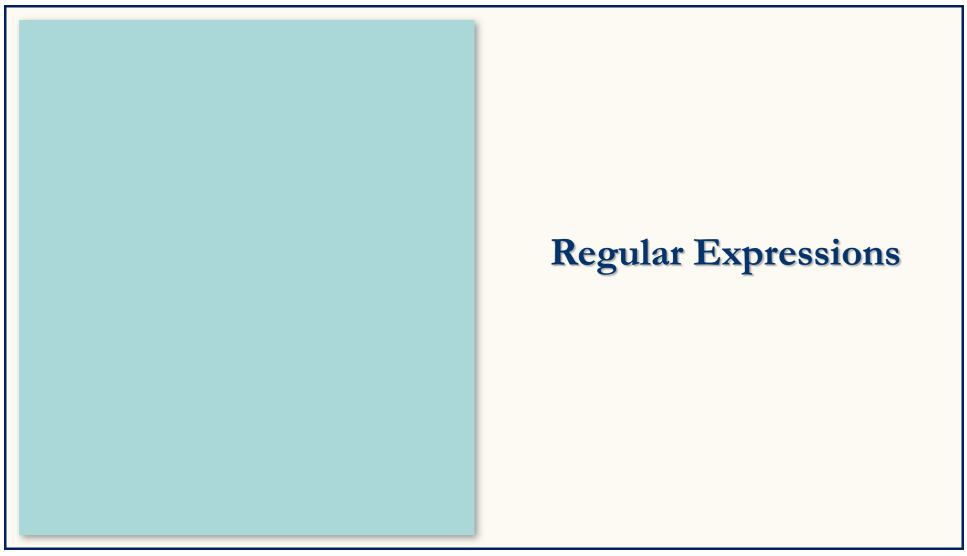
- Simple regex matching in Python is done via the `re` module's `match` or `search` functions.

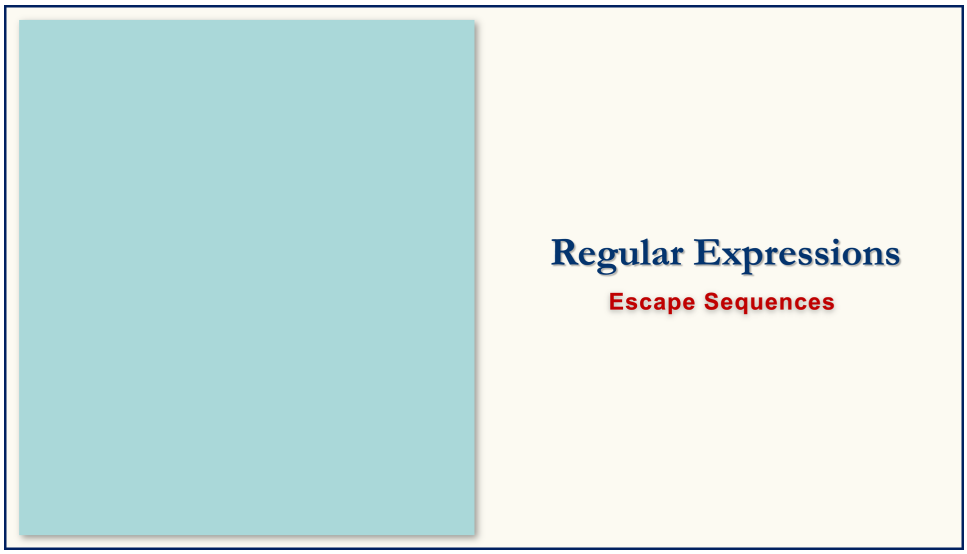
- This lecture introduces regex awareness; advanced validations require further constructs like assertions and conditional logic.

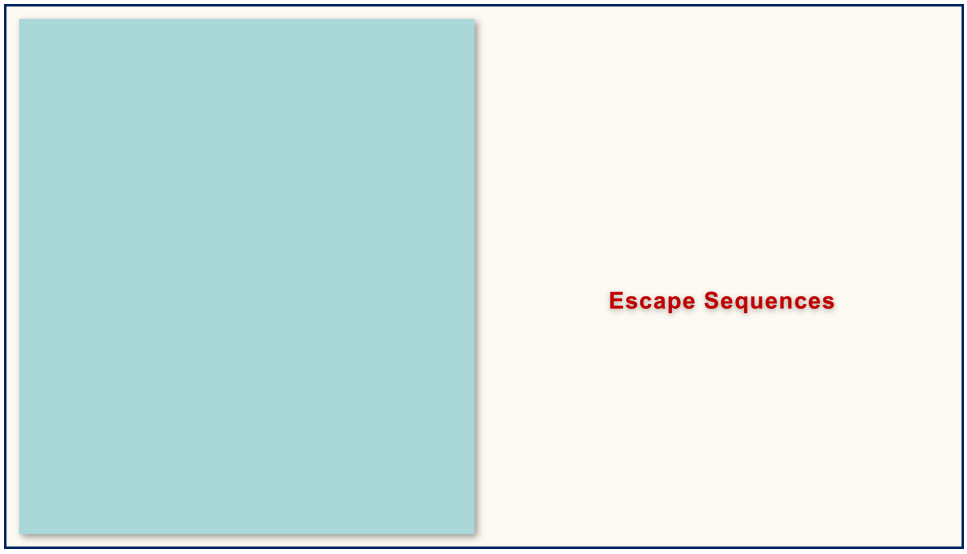
- Practice building and testing regex patterns to become comfortable with these concepts.

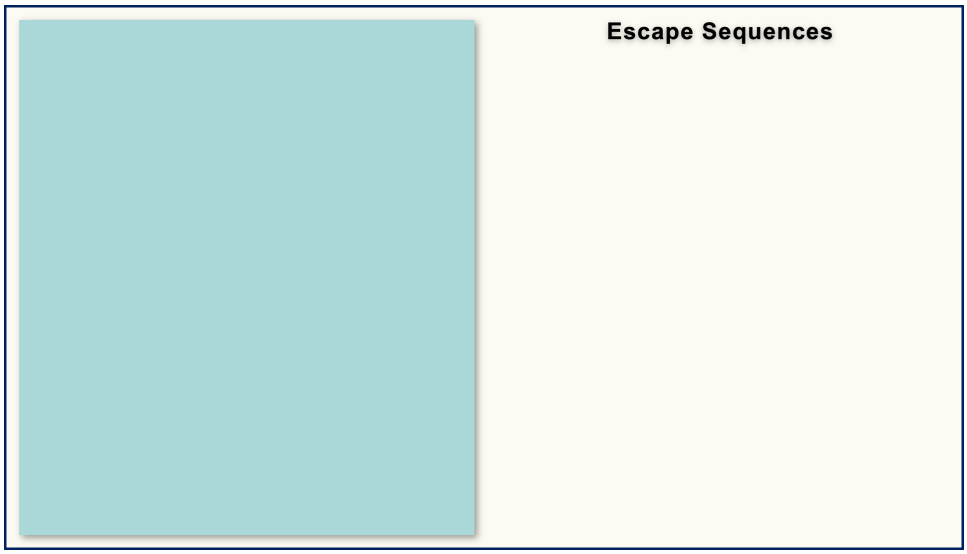
Lecture in Slides

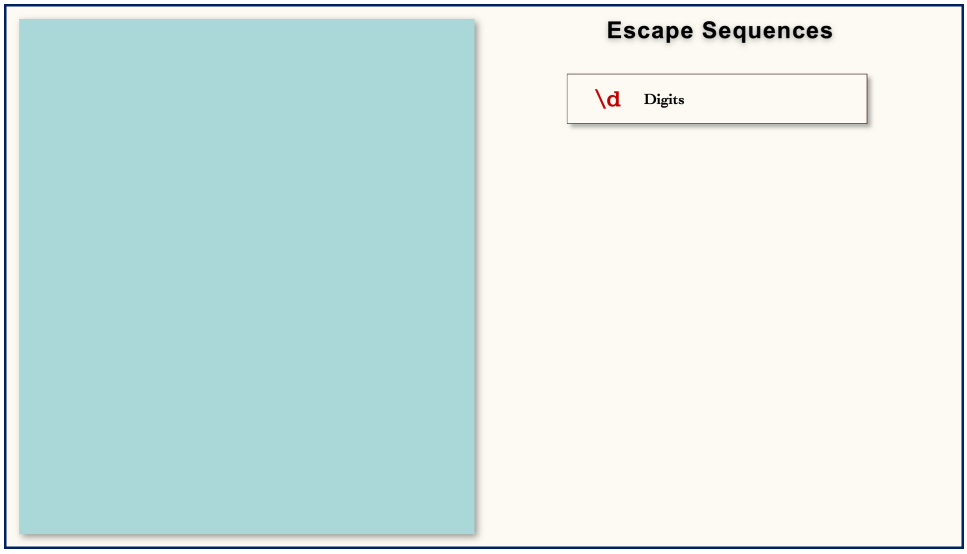
Please Note: This document presents a curated selection of slides from the lecture; others have been excluded for conciseness.

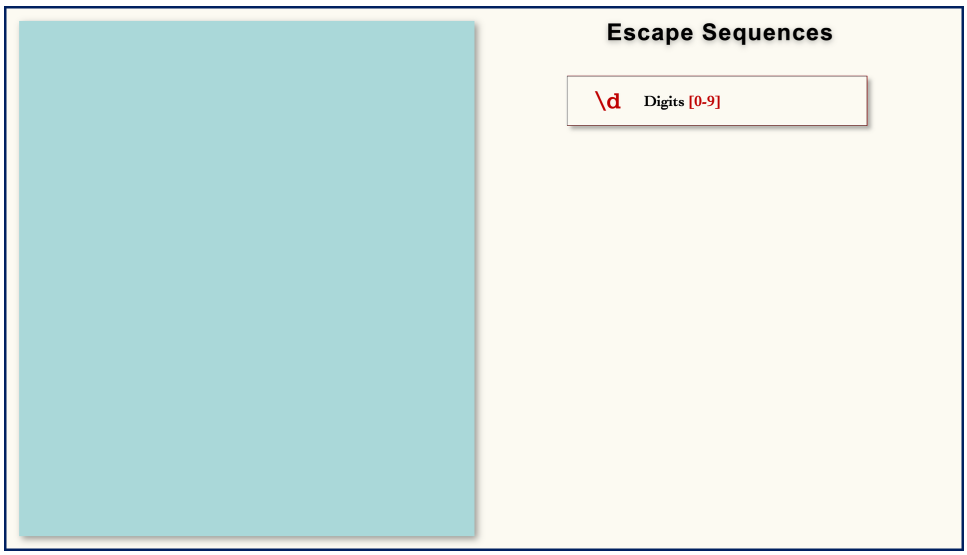


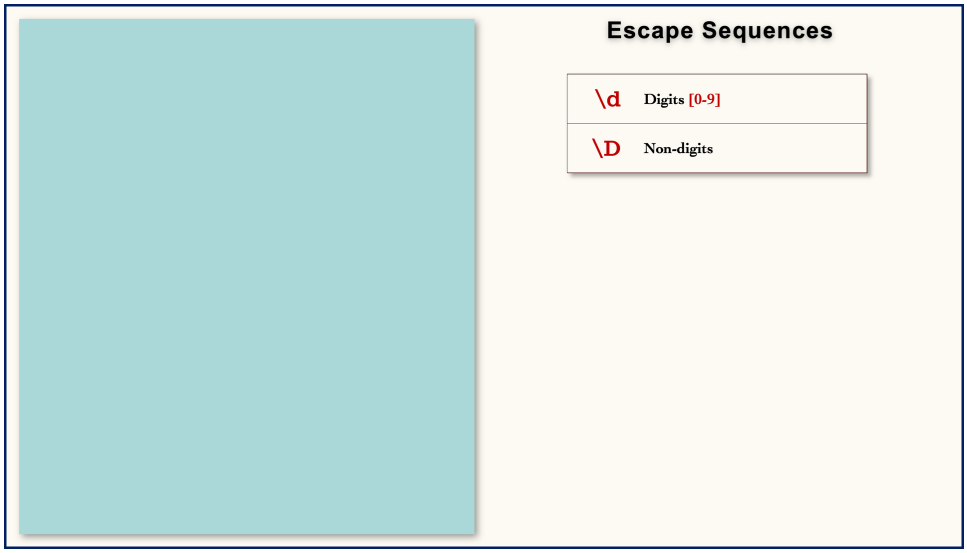

Regular Expressions
Escape Sequences

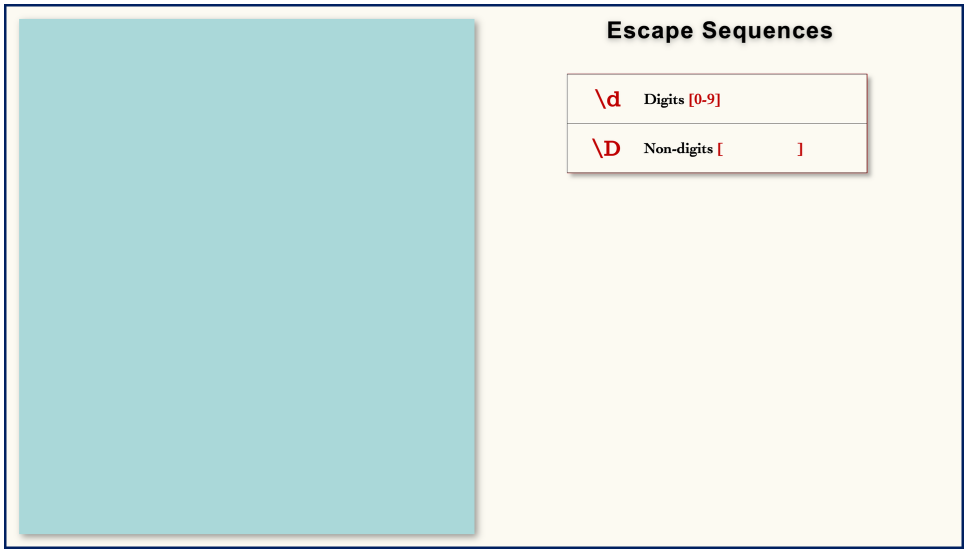

Escape Sequences

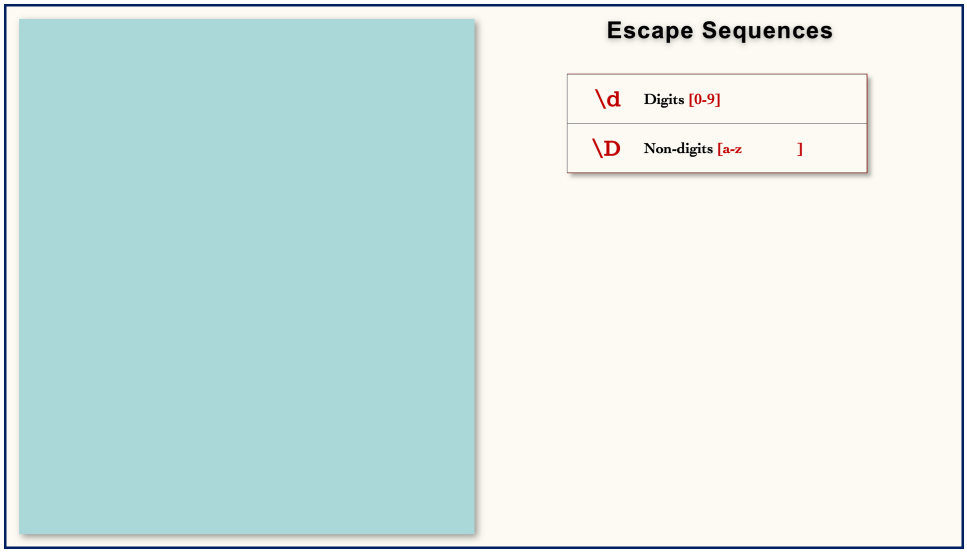

Escape Sequences

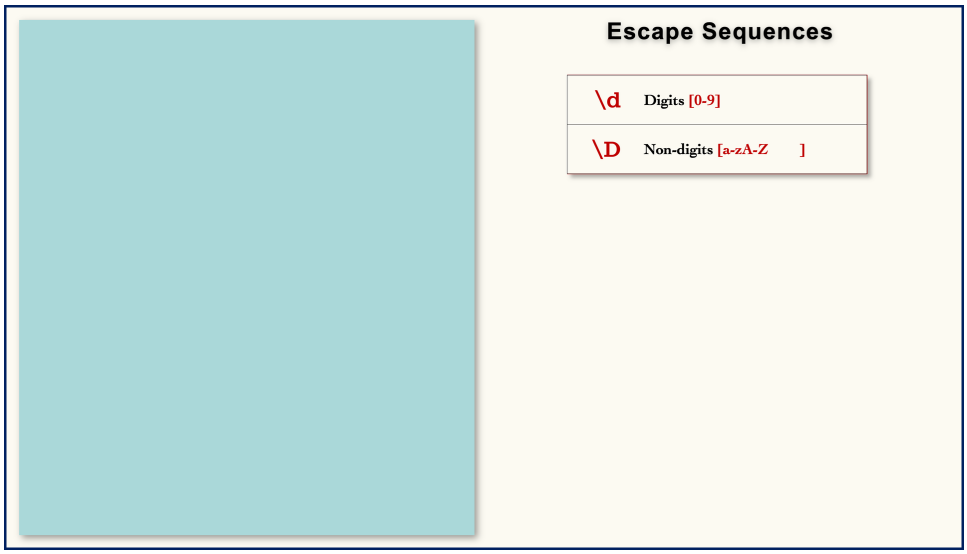

Escape Sequences
\d
Digits

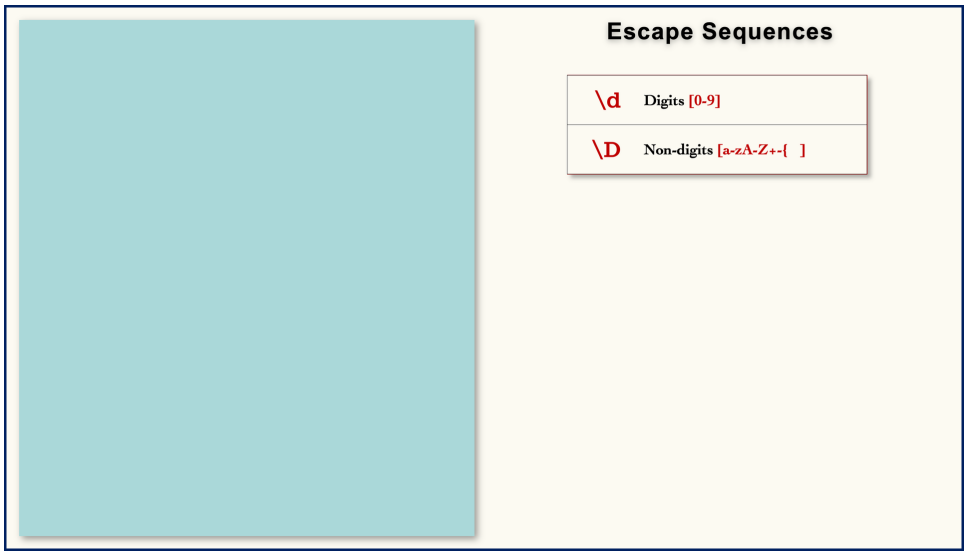

Escape Sequences
\d
Digits [0-9]

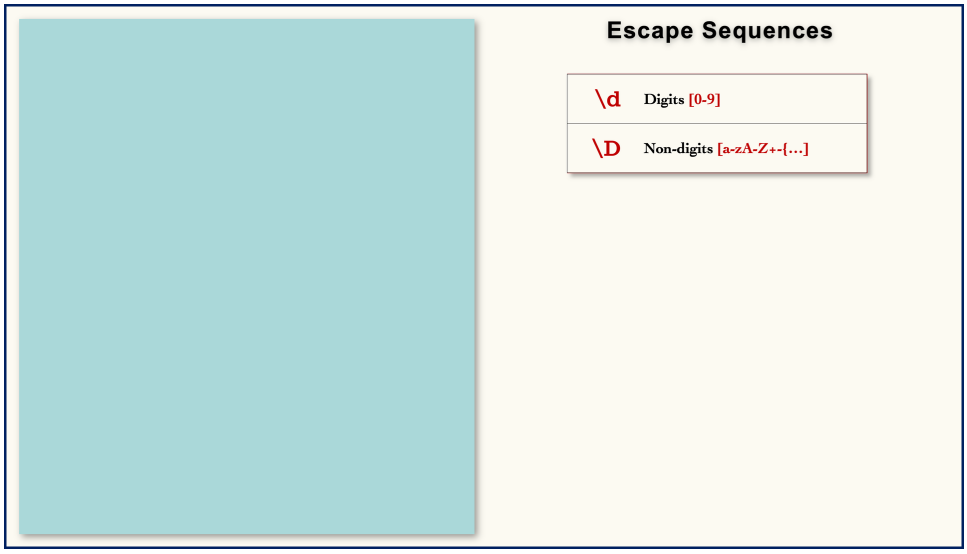

Escape Sequences
\d
Digits [0-9]
\D
Non-digits

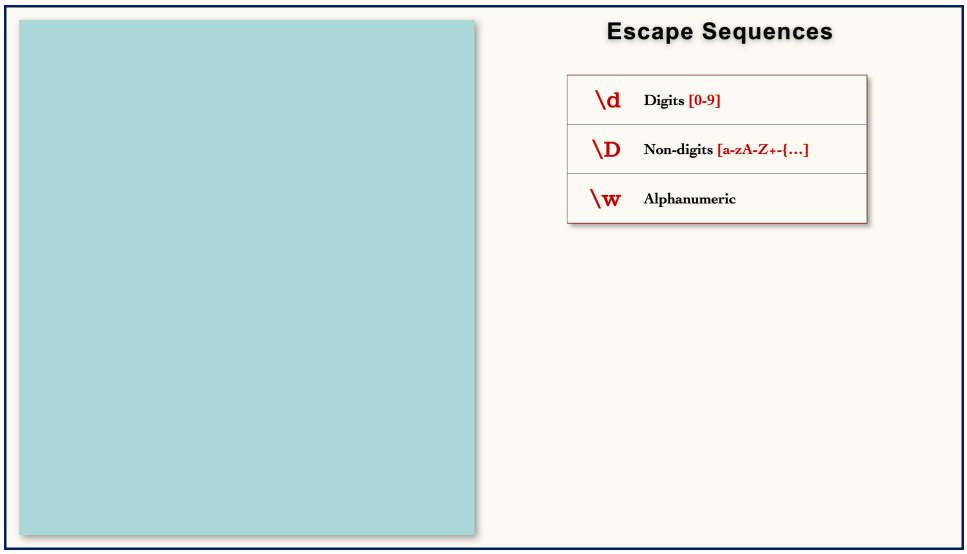

Escape Sequences
\d Digits [0-9]
\D Non-digits [ ]

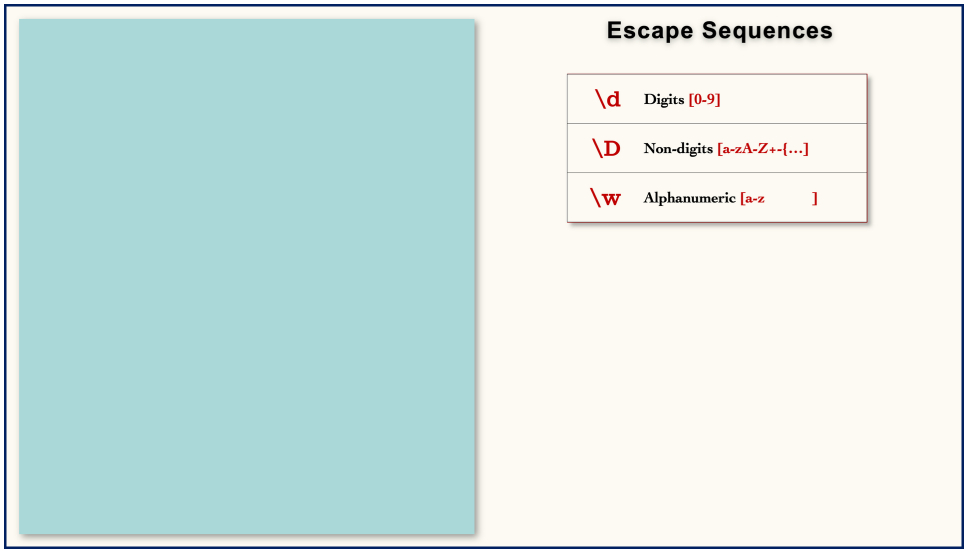

Escape Sequences
\d Digits [0-9]
\D Non-digits [a-z]

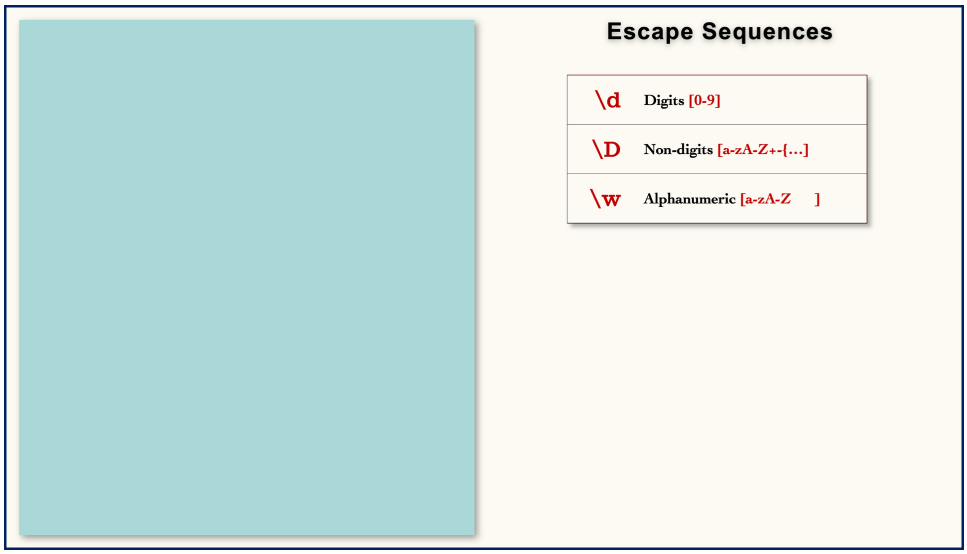

Escape Sequences
\d Digits [0-9]
\D Non-digits [a-zA-Z]

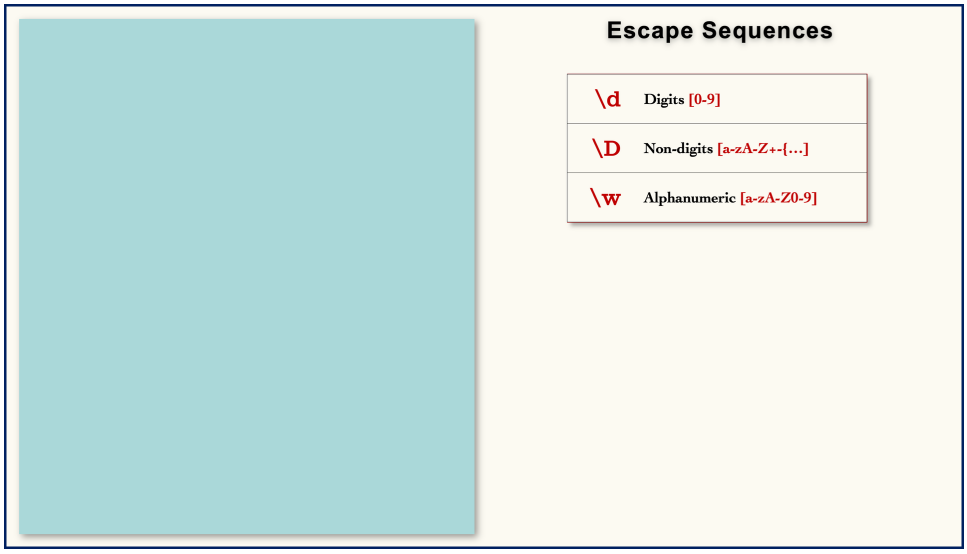

Escape Sequences
\d Digits [0-9]
\D Non-digits [a-zA-Z+-{]

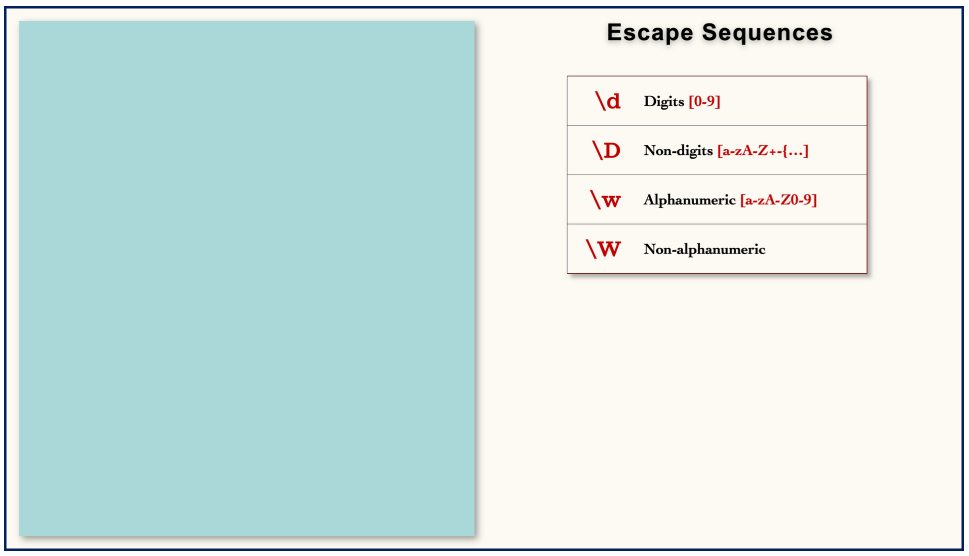

Escape Sequences
\d
Digits [0-9]
\D
Non-digits [a-zA-Z+-[…]

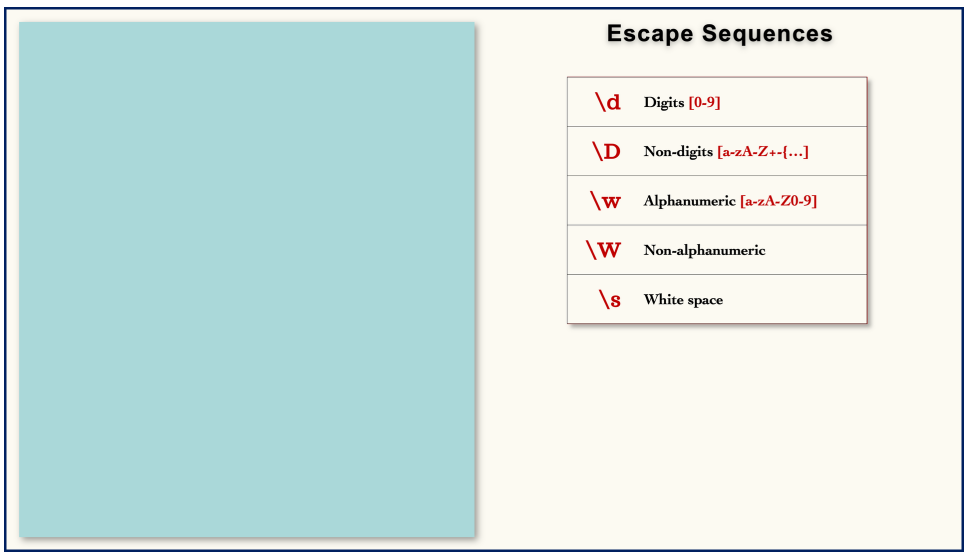

Escape Sequences
\d Digits [0-9]
\D Non-digits [a-zA-Z+-{…]
\w Alphanumeric

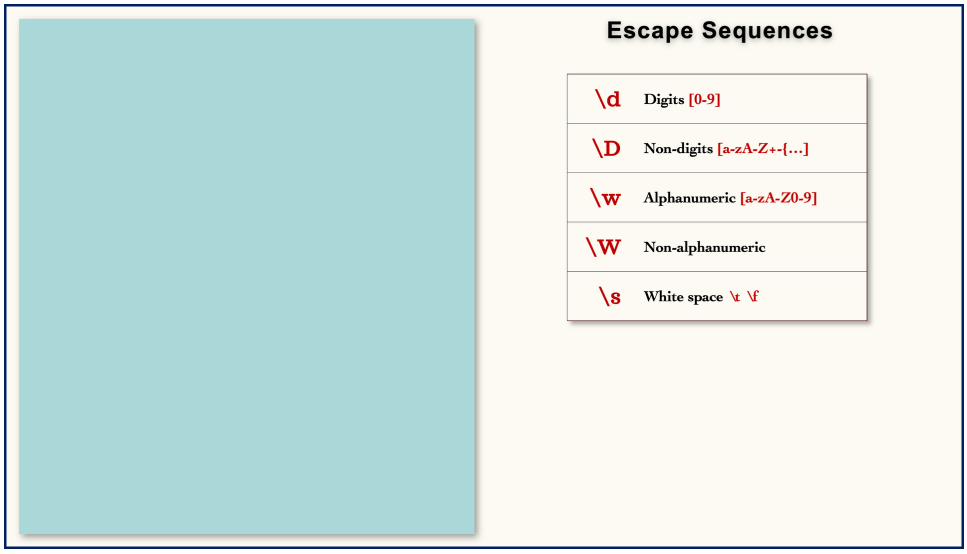

Escape Sequences
\d Digits [0-9]
\D Non-digits [a-zA-Z+-{…]
\w Alphanumeric [a-z]

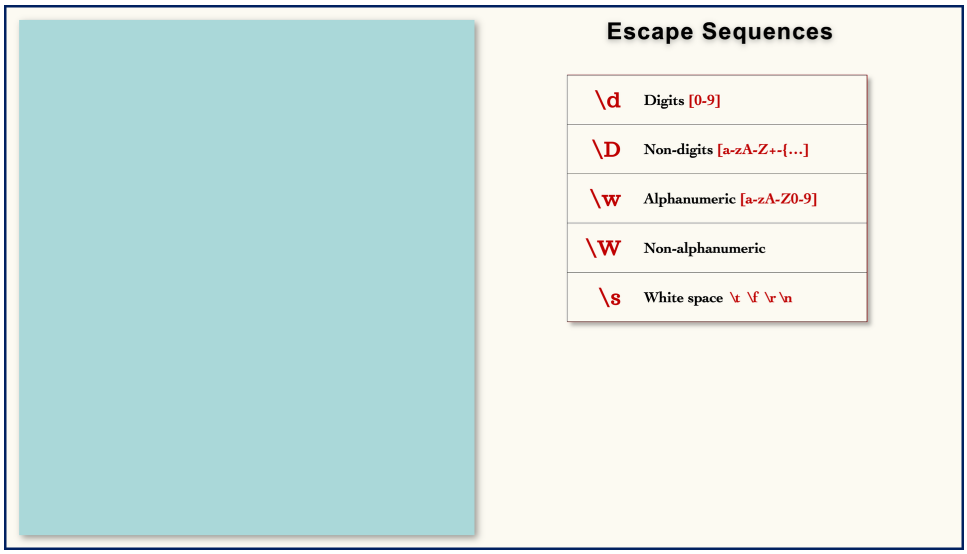

Escape Sequences
\d Digits [0-9]
\D Non-digits [a-zA-Z+-{…]
\w Alphanumeric [a-zA-Z]


Escape Sequences
\d Digits [0-9]
\D Non-digits [a-zA-Z+-{…]
\w Alphanumeric [a-zA-Z0-9]

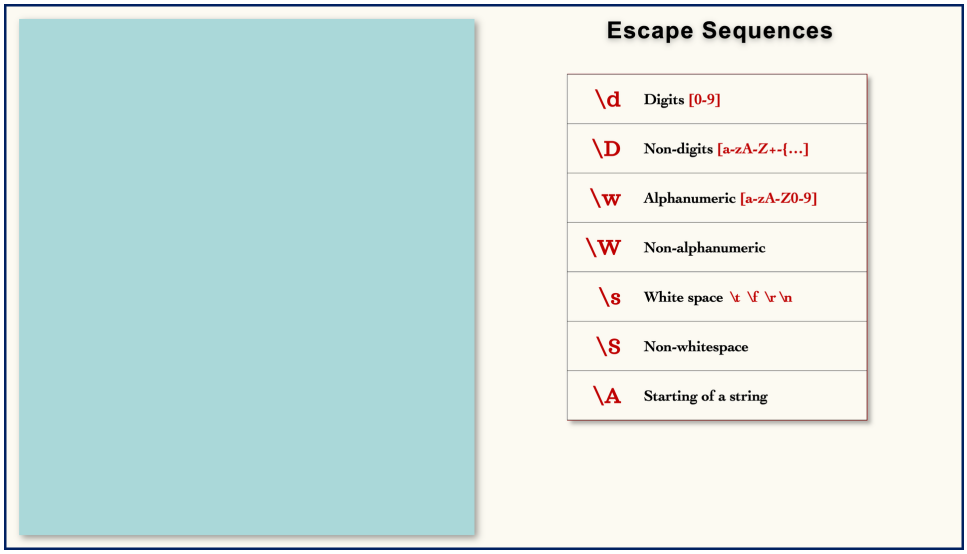

Escape Sequences
\d Digits [0-9]
\D Non-digits [a-zA-Z+-{…]
\w Alphanumeric [a-zA-Z0-9]
\W Non-alphanumeric

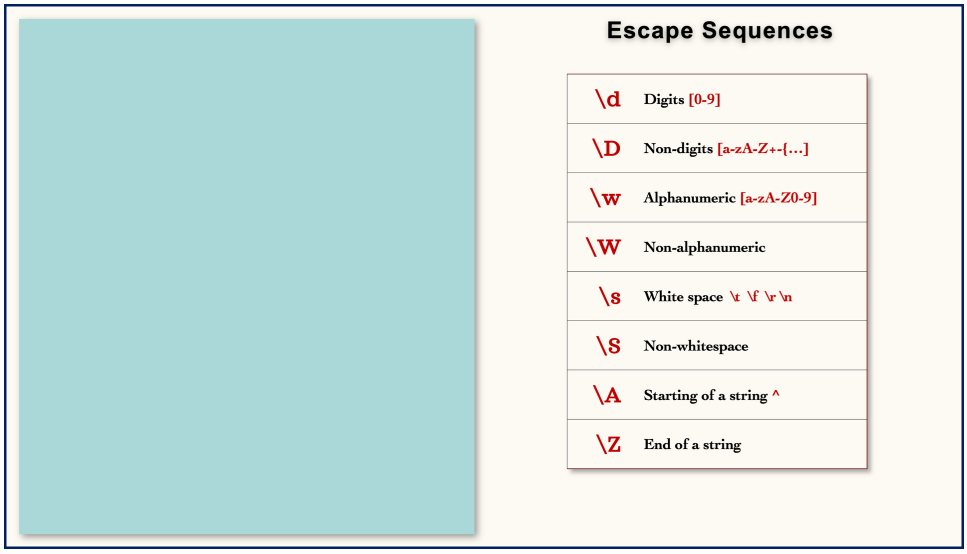

Escape Sequences
\d Digits [0-9]
\D Non-digits [a-zA-Z+-{…]
\w Alphanumeric [a-zA-Z0-9]
\W Non-alphanumeric
\s White space

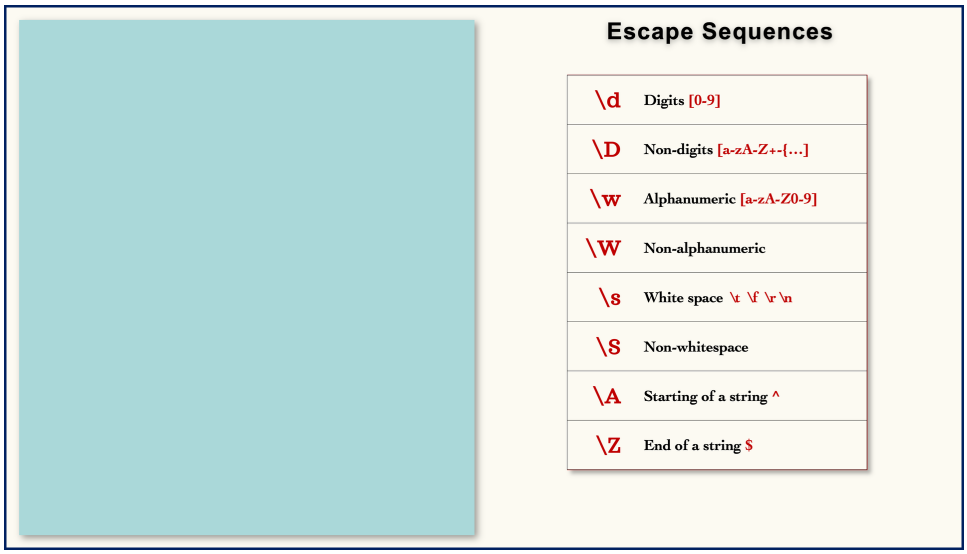

Escape Sequences
\d Digits [0-9]
\D Non-digits [a-zA-Z+-[…]
\w Alphanumeric [a-zA-Z0-9]
\W Non-alphanumeric
\s White space \t \f

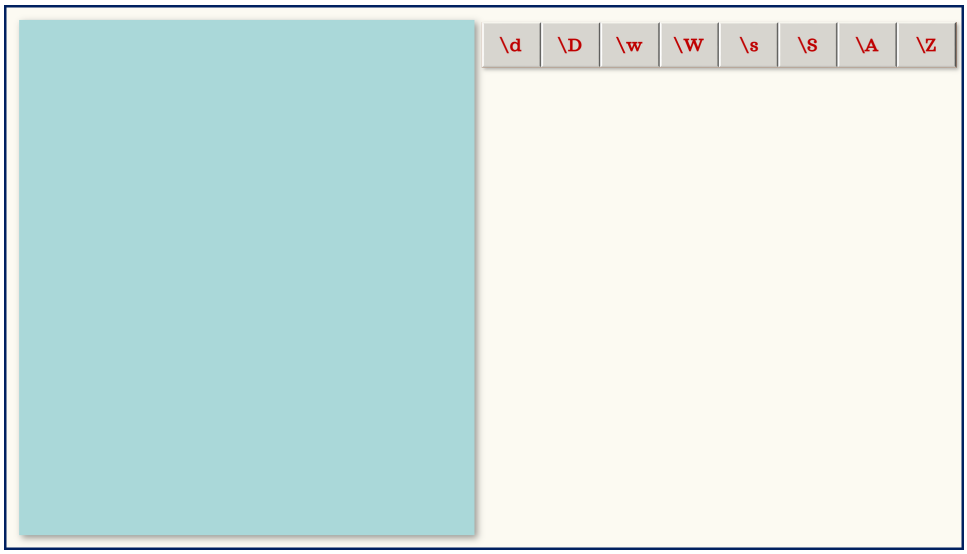

Escape Sequences
\d Digits [0-9]
\D Non-digits [a-zA-Z+-[…]
\w Alphanumeric [a-zA-Z0-9]
\W Non-alphanumeric
\s White space \t \f \r \n

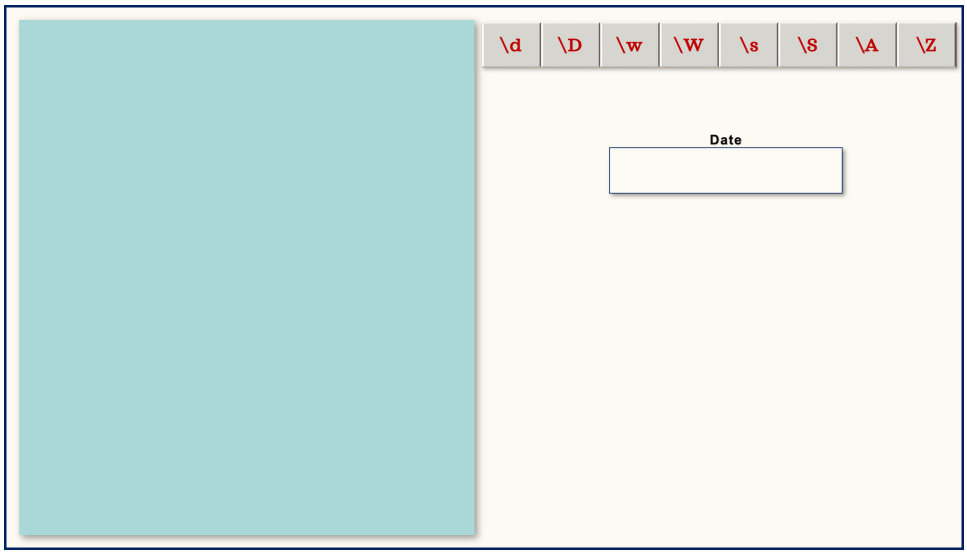

Escape Sequences
\d Digits [0-9]
\D Non-digits [a-zA-Z+-[…]
\w Alphanumeric [a-zA-Z0-9]
\W Non-alphanumeric
\s White space \t \f \r \n
\S Non-whitespace

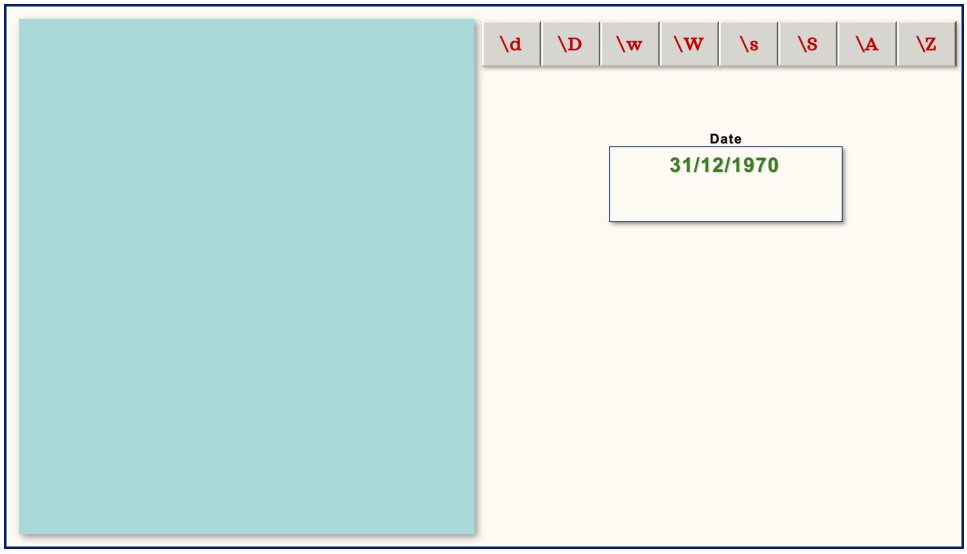

Escape Sequences
\d Digits [0-9]
\D Non-digits [a-zA-Z+-[…]
\w Alphanumeric [a-zA-Z0-9]
\W Non-alphanumeric
\s White space \t \f \r \n
\S Non-whitespace
\A Starting of a string

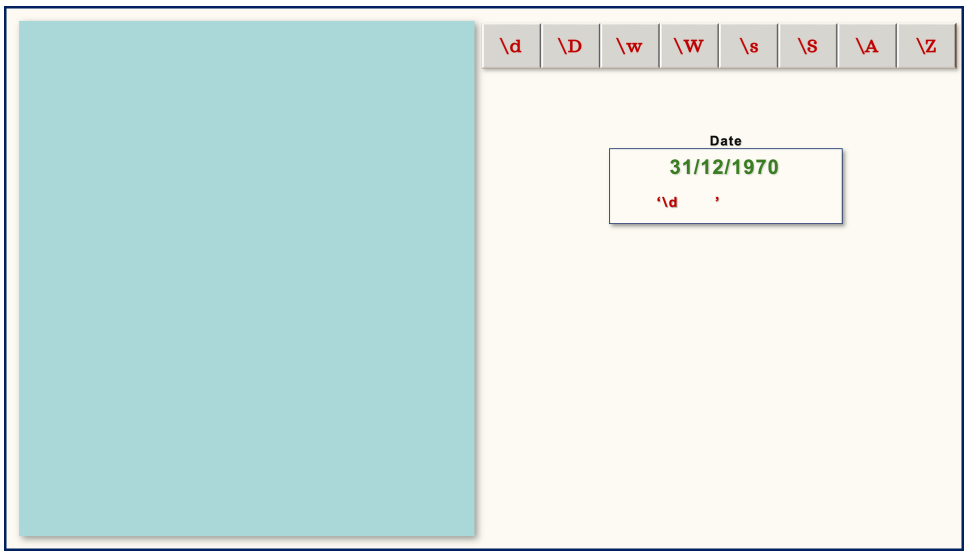

Escape Sequences
\d Digits [0-9]
\D Non-digits [a-zA-Z+-[…]
\w Alphanumeric [a-zA-Z0-9]
\W Non-alphanumeric
\s White space \t \f \r \n
\S Non-whitespace
\A Starting of a string ^
\Z End of a string

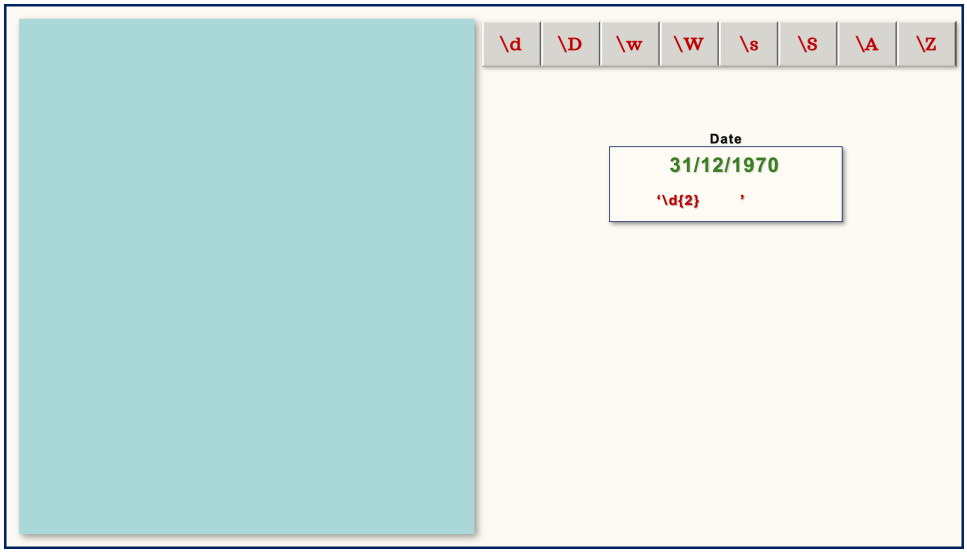

Escape Sequences
\d Digits [0-9]
\D Non-digits [a-zA-Z+-{...]
\w Alphanumeric [a-zA-Z0-9]
\W Non-alphanumeric
\s White space \t \f \r \n
\S Non-whitespace
\A Starting of a string ^
\Z End of a string $

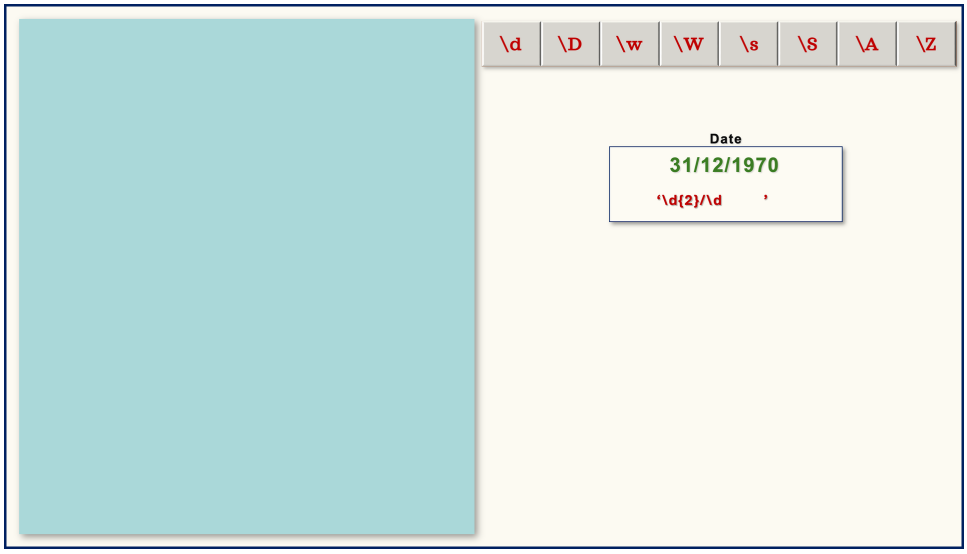

\d
\D
\w
\W
\s
\S
\A
\Z

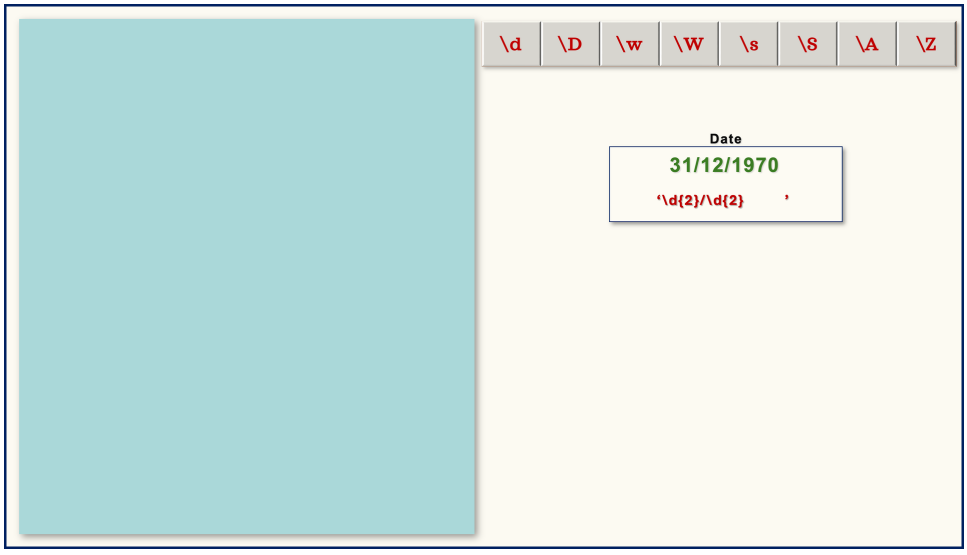

\d
\D
\w
\W
\s
\S
\A
\Z
Date

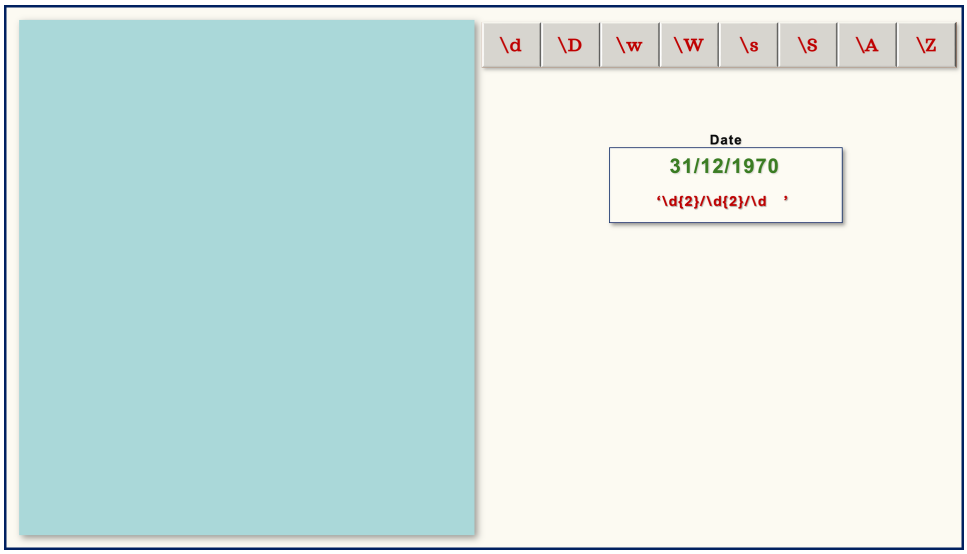

\d
\D
\w
\W
\s
\S
\A
\Z
Date
31/12/1970

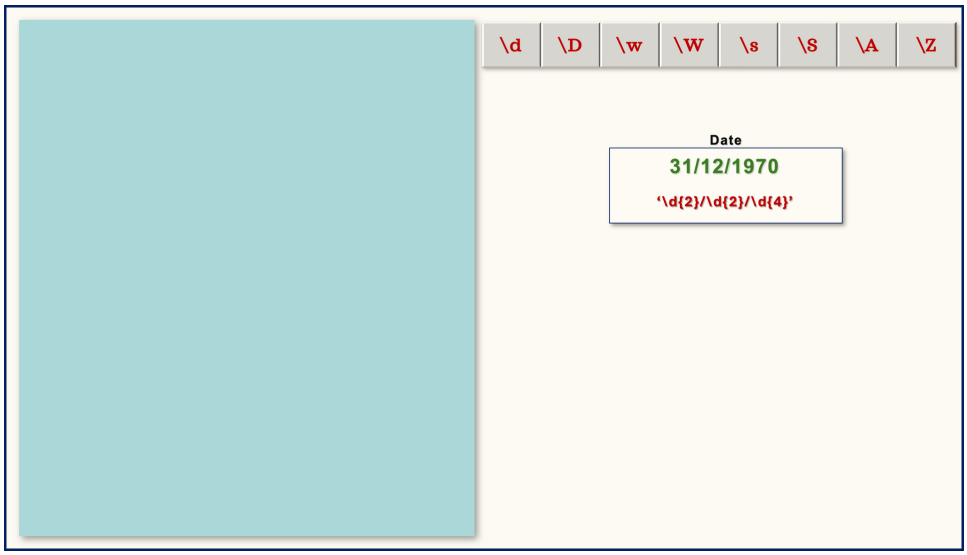

\d
\D
\w
\W
\s
\S
\A
\Z
Date
31/12/1970
'\d '

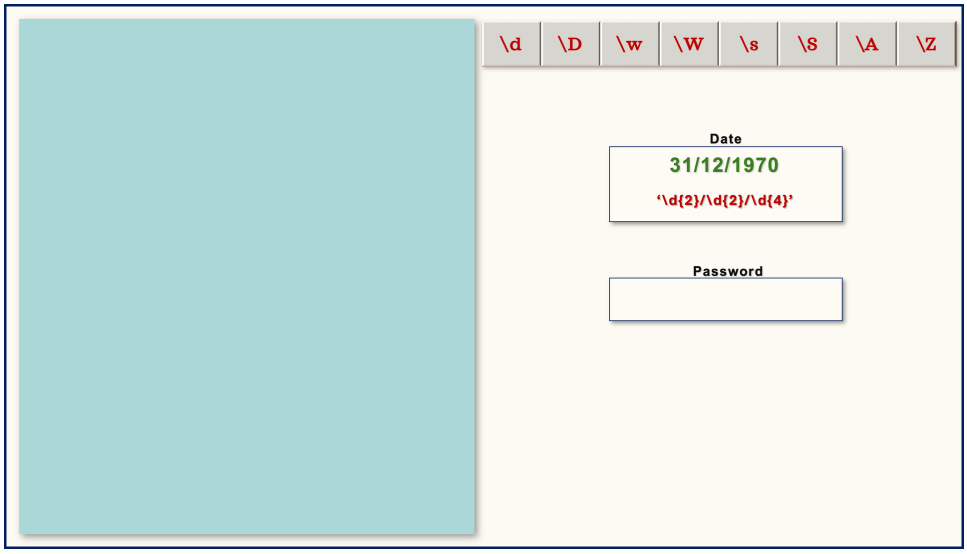

\d
\D
\w
\W
\s
\S
\A
\Z
Date
31/12/1970
'\d{2} '

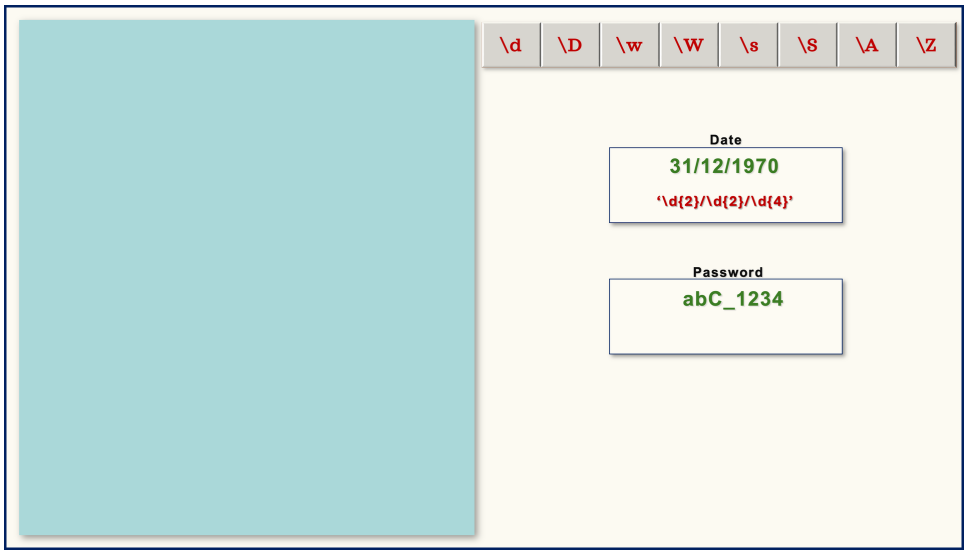

\d
\D
\w
\W
\s
\S
\A
\Z
Date
31/12/1970
'\d{2}/\d '

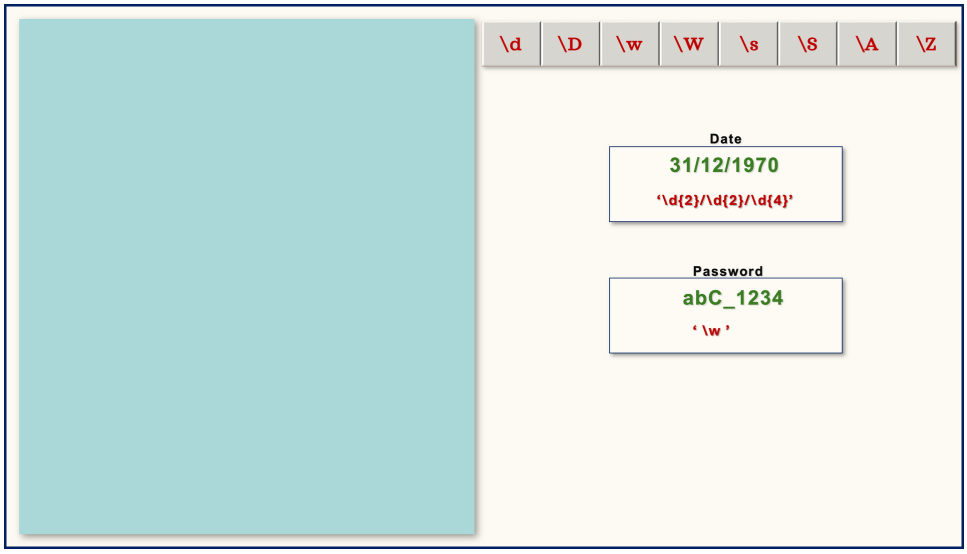

\d
\D
\w
\W
\s
\S
\A
\Z
Date
31/12/1970
'\d{2}/\d{2} '

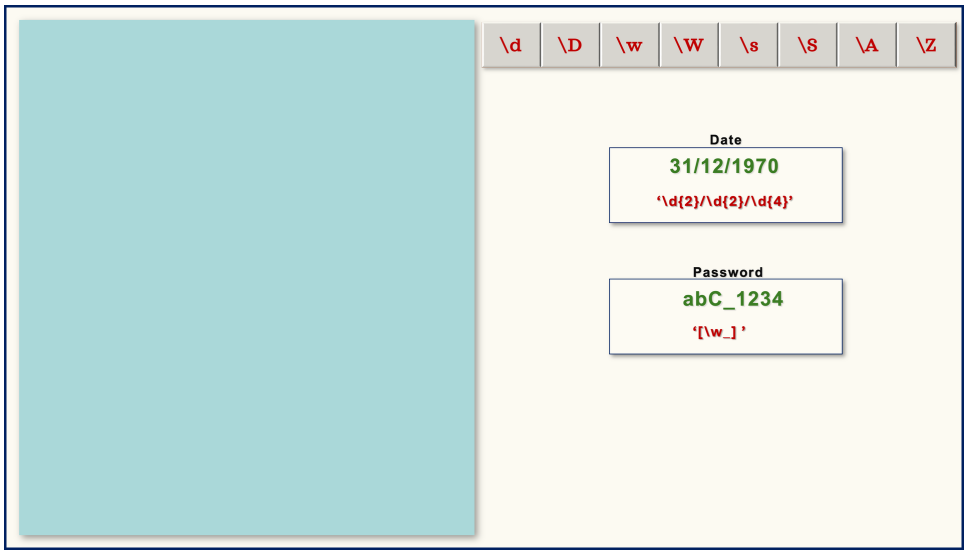

\d
\D
\w
\W
\s
\S
\A
\Z
Date
31/12/1970
'\d{2}/\d{2}/\d '

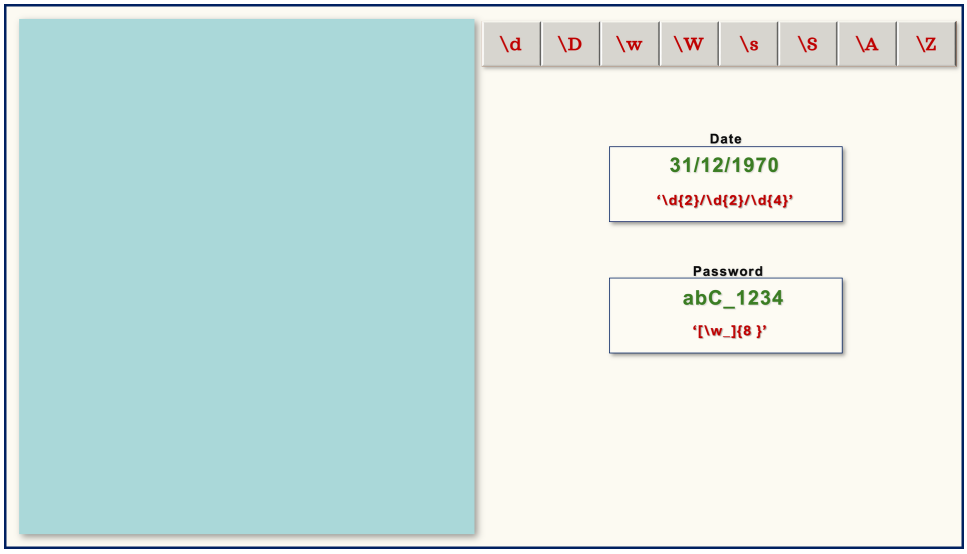

\d
\D
\w
\W
\s
\S
\A
\Z
Date
31/12/1970
'\d{2}/\d{2}/\d{4}'

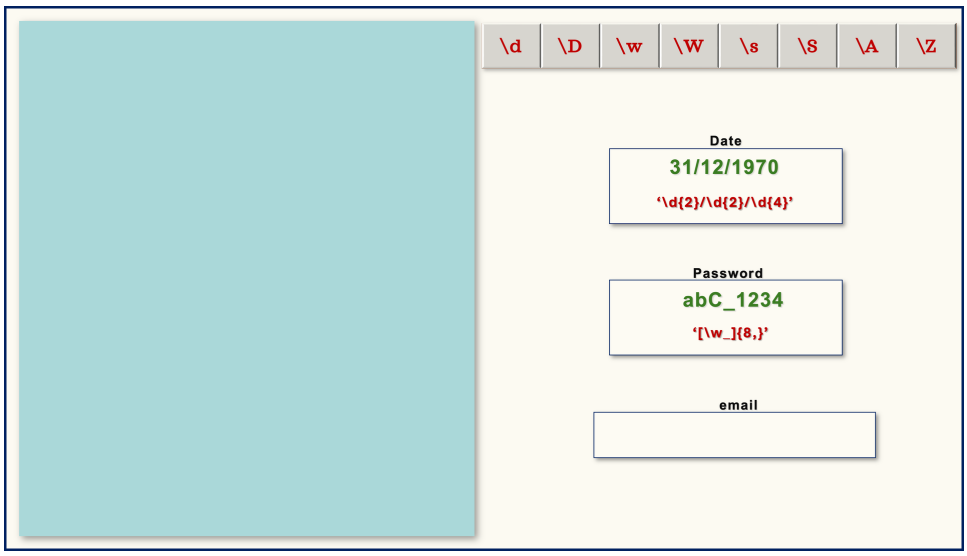

\d
\D
\w
\W
\s
\S
\A
\Z
Date
31/12/1970
'\d{2}/\d{2}/\d{4}'
Password

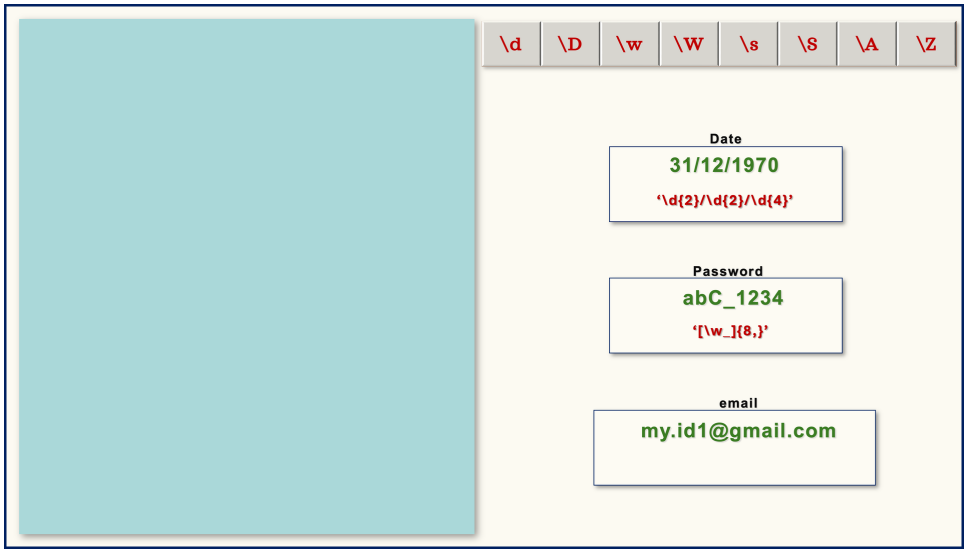

\d
\D
\w
\W
\s
\S
\A
\Z
Date
31/12/1970
'\d{2}/\d{2}/\d{4}'
Password
abC_1234

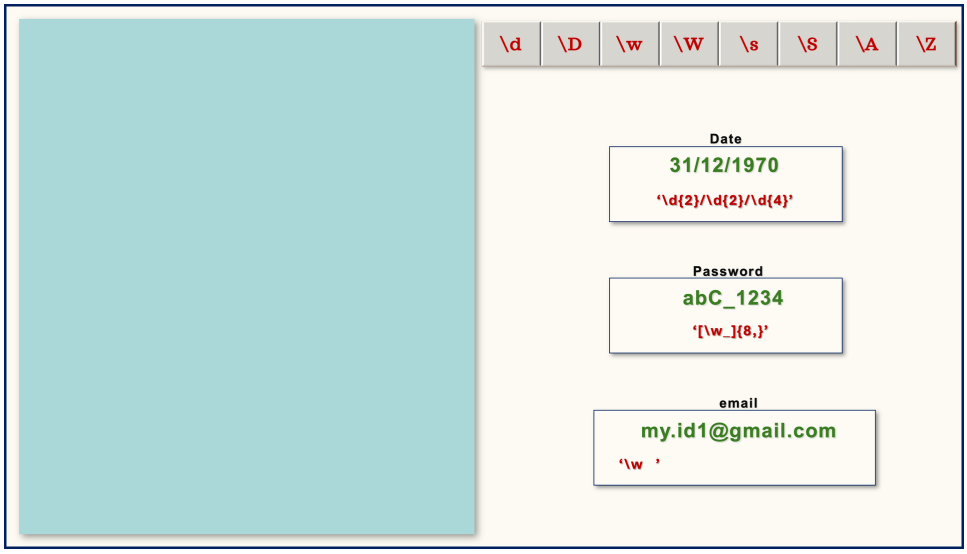

\d
\D
\w
\W
\s
\S
\A
\Z
Date
31/12/1970
'\d{2}/\d{2}/\d{4}'
Password
abC_1234
' \w '

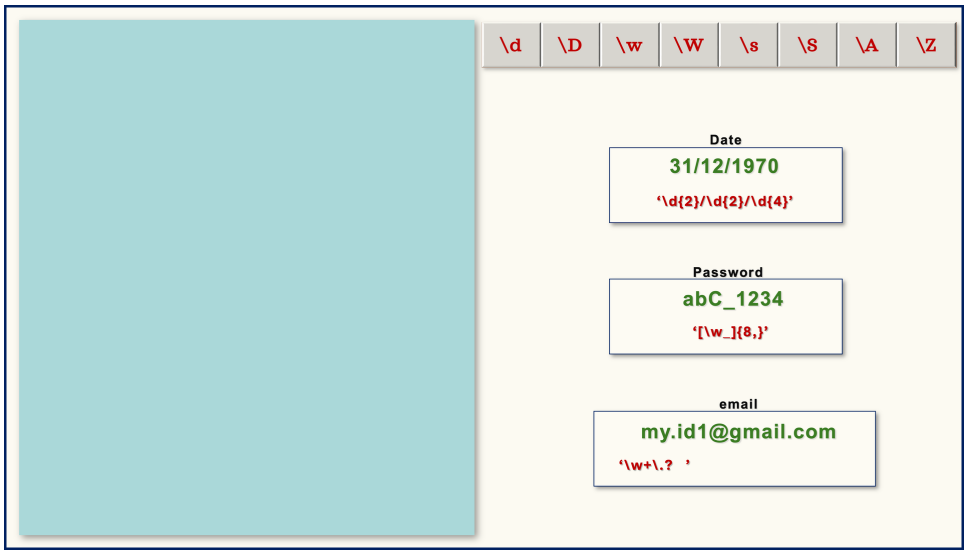

\d
\D
\w
\W
\s
\S
\A
\Z
Date
31/12/1970
'\d{2}/\d{2}/\d{4}'
Password
abC_1234
'[\w_] '

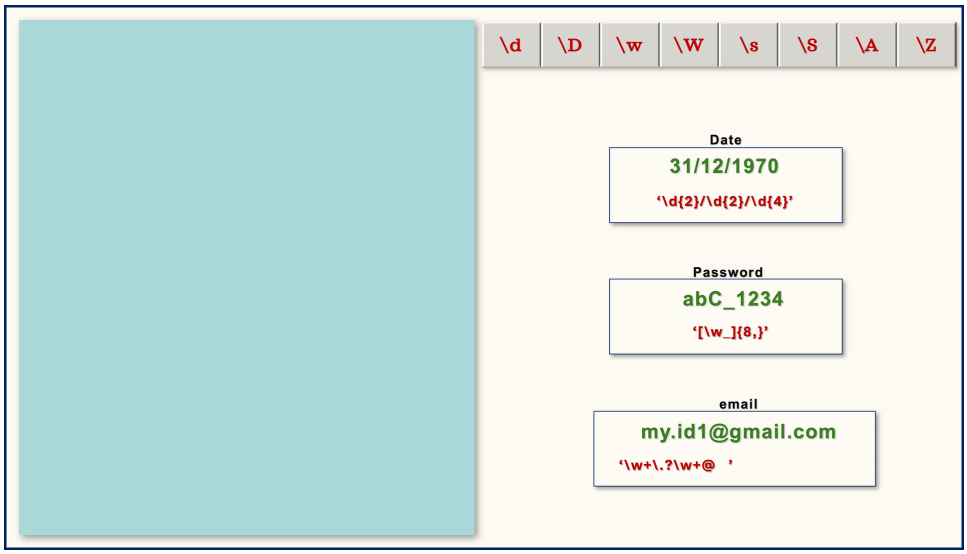

\d
\D
\w
\W
\s
\S
\A
\Z
Date
31/12/1970
'\d{2}/\d{2}/\d{4}'
Password
abC_1234
'[\w_]{8 }'

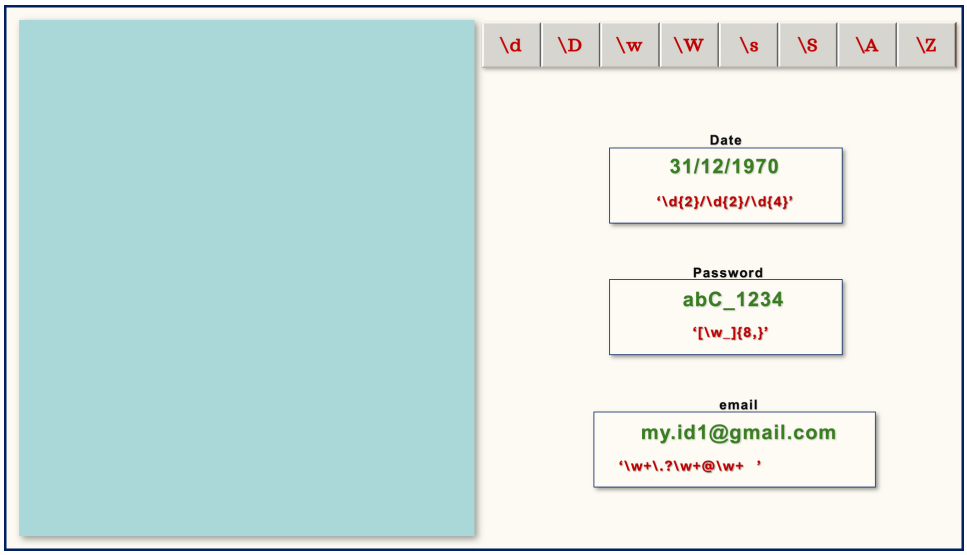

\d
\D
\w
\W
\s
\S
\A
\Z
Date
31/12/1970
'\d{2}/\d{2}/\d{4}'
Password
abC_1234
'[\w_]{8,}'
email

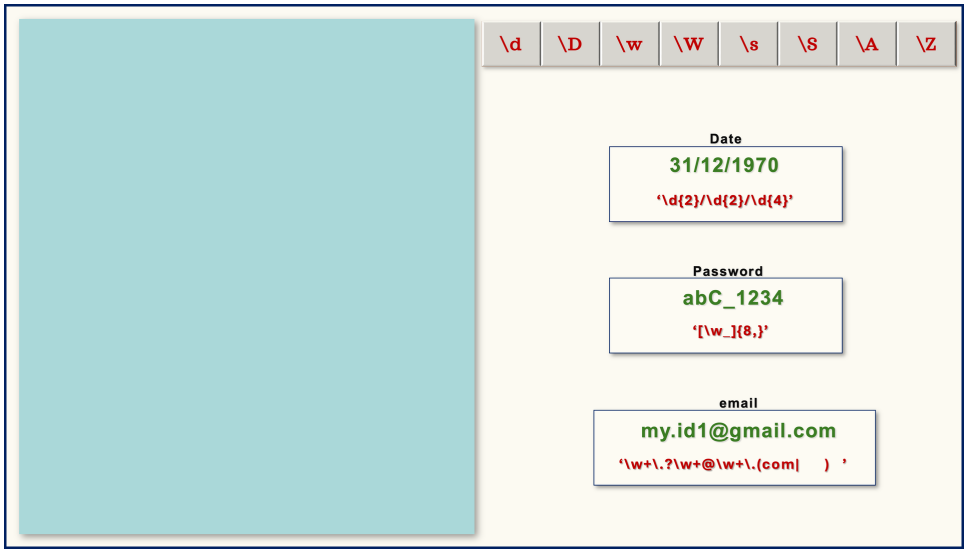

\d
\D
\w
\W
\s
\S
\A
\Z
Date
31/12/1970
'\d{2}/\d{2}/\d{4}'
Password
abC_1234
'[\w_]{8,}'
email
my.id1@gmail.com

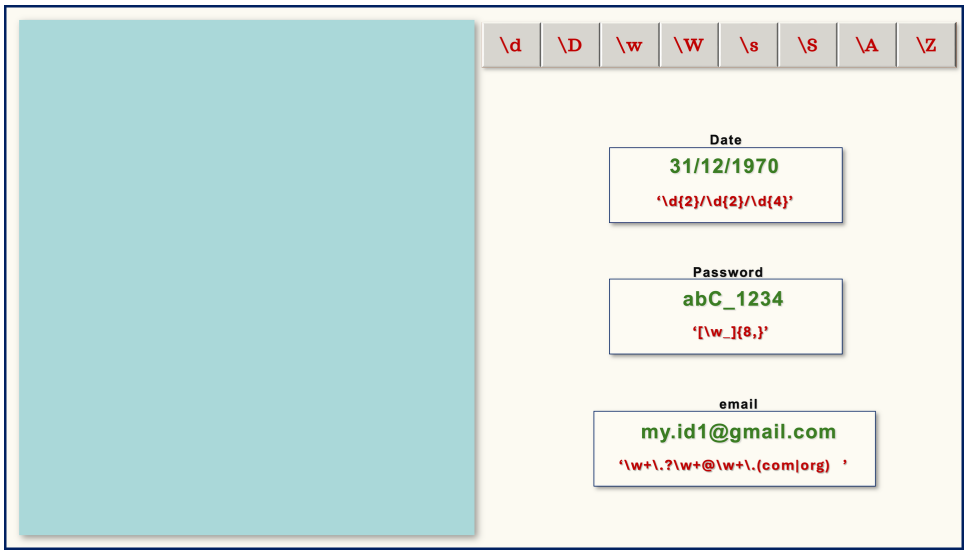

\d
\D
\w
\W
\s
\S
\A
\Z
Date
31/12/1970
'\d{2}/\d{2}/\d{4}'
Password
abC_1234
'[\w_]{8,}'
email
my.id1@gmail.com
'\w '

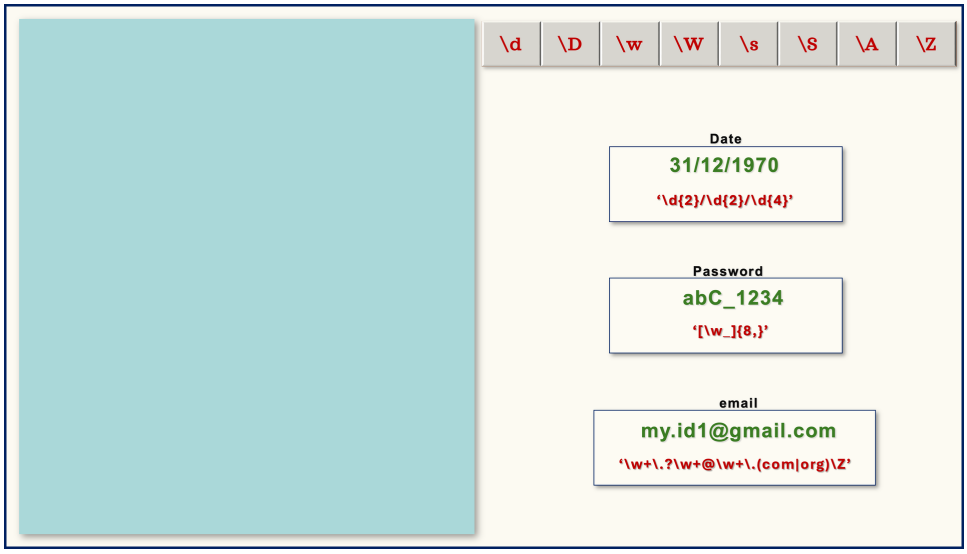

\d
\D
\w
\W
\s
\S
\A
\Z
Date
31/12/1970
'\d{2}/\d{2}/\d{4}'
Password
abC_1234
'[\w_]{8,}'
email
my.id1@gmail.com
'\w+\.? '


\d
\D
\w
\W
\s
\S
\A
\Z
Date
31/12/1970
'\d{2}/\d{2}/\d{4}'
Password
abC_1234
'[\w_]{8,}'
email
my.id1@gmail.com
'\w+\.?\w+@ '


\d
\D
\w
\W
\s
\S
\A
\Z
Date
31/12/1970
'\d{2}/\d{2}/\d{4}'
Password
abC_1234
'[\w_]{8,}'
email
my.id1@gmail.com
'\w+\.?\w+@\w+ '


\d
\D
\w
\W
\s
\S
\A
\Z
Date
31/12/1970
'\d{2}/\d{2}/\d{4}'
Password
abC_1234
'[\w_]{8,}'
email
my.id1@gmail.com
'\w+\.?\w+@\w+\.(com|) '


\d
\D
\w
\W
\s
\S
\A
\Z
Date
31/12/1970
'\d{2}/\d{2}/\d{4}'
Password
abC_1234
'[\w_]{8,}'
email
my.id1@gmail.com
'\w+\.?\w+@\w+\.(com|org) '


\d
\D
\w
\W
\s
\S
\A
\Z
Date
31/12/1970
'\d{2}/\d{2}/\d{4}'
Password
abC_1234
'[\w_]{8,}'
email
my.id1@gmail.com
'\w+\.?\w+@\w+\.(com|org)\Z'